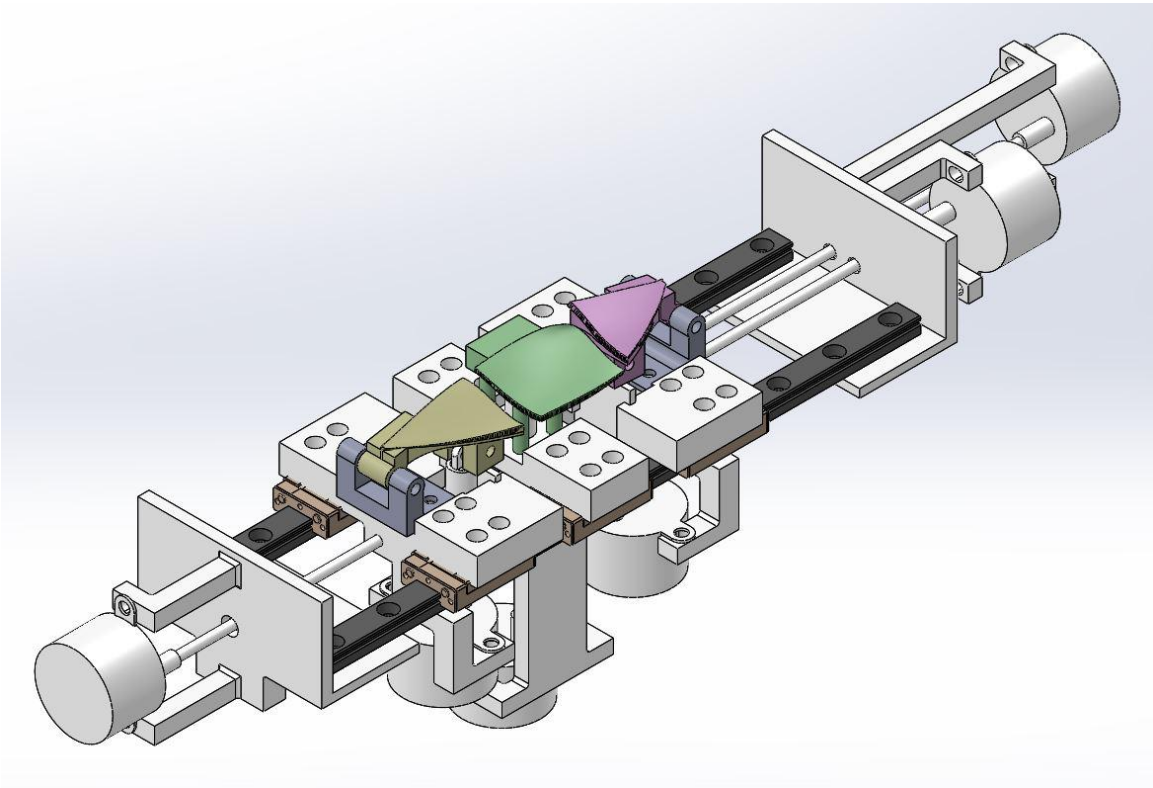
2019.8.6

Group 9 G.O.A.T.

SmartArch: Foot Orthoses Shape Fitting System



Yiding Chang

Zhongqian Duan

Chen Huang

Lang Shen

Haotian Xie

Table of Contents

I. EXECUTIVE SUMMARY....3

II. ACKNOWLEDGEMENTS....4

III. PROBLEM....5

IV. NEEDS....7

V. SOLUTION: SMARTARCH....8

VI. OBJECTIVES....10

VII. TASKS....11

VIII. DISCUSSION....21

IX. CONCLUSION....22

X. SCHEDULE....23

XI. BILL OF MATERIALS....24

XII. KEY PERSONNEL....26

XIII. REFERENCES....27

APPENDIX A: THE DATA OF THE LINEAR FIT FOR THE PRESSURE SENSOR....28

APPENDIX B: THE CORRESPONDING SIGNAL OF THE IR REMOTE CONTROL'S BUTTONS....30

APPENDIX C: FULL CODES....31

I. Executive Summary

Flat foot, as well as many other severe feet illnesses, is caused by the deformation and extortion of soft tissues and bones. Once the foot arch is damaged, patients will have difficulty completing even basic movements like walking. Although the application of orthoses or other foot protection devices are widely used, it takes some thousands of dollars to measure the feet arch in detail using 3D scanning. In addition, pressure, which is a vital factor in healing the foot arch, is ignored in the scanning method. Our answer to this question is a new system that can measure the feet arch under pressure. It is a simple device with three segments that can record the shape of the foot arch and pressure it applies on the orthoses. To achieve this goal, we managed to complete some tasks and realized our functions step by step. Initially we used SolidWorks to establish our first 3D-design of the prototype of the equipment. Then we 3D-printed the components and assembled them together. In addition, we used stepper motors to move these segments, and algorithms were designed so that the device will fit the foot arch automatically. Our plan was validated, as the segments can move accurately according to the order from the remote control. When the foot is placed on the measurement device, it automatically supported the foot arch and the screen showed the pressure that applied on the foot arch. However, it can still use some improvements, as the segments' movement can be accelerated. Conclusively, our measurement device effectively cut down the cost of measuring foot arch and can be used widely in the medical fields.

II. Acknowledgements

Thanks to Dr. Shane Johnson and Prof. Michele Campbell for their guidance on technical and tech-communication parts respectively.

Thanks to Mr. Siwei Ye and Mr. Haichao Yang for their patience when answering our questions about the project.

Thanks to Mr. Tian Gao and Miss Yating Liu for their advice on our slides, poster and report.

Thanks to Mr. Otto Zhang for his patience when letting us use the lab.

Thanks to Dr. Shane Johnson and Zeeshan Qaiser for providing us with this idea and permitting us to work on it.

Thanks to every group number for their hard work.

Special thanks to:

Mr. Chang's magnificent leadership,

Mr. Huang's genuine inspiration,

Mr. Duan's extraordinary contribution,

Mr. Shen's warm-heartedness,

and Mr. Xie's dedication.

Excellent work!

III. Problem

Figure 3.1 The deformation of foot bones leads to flat foot, increasing the risk of being hurt [2] [3]

Flat foot problem appears when the structure of the foot bones is deformed, leading to the disappearance of the foot arch, as is shown in Figure 3.1. Without the support of the foot arch, the ankle will deform, and bear a higher risk of getting sprained. The patient's whole body might experience some change: X-shape legs, twisted spines, etc. What's more, the rate of having the flat foot problem is very high. A survey taken in Hong Kong including 474 students between 6 and 10 years old found out that up to 22.4% of students have the flat foot problem [1].

Foot orthoses have been proved effective in treating the flat foot problem and its complicating diseases [4]. The existing method to measure a person's foot arch and build orthoses is using a 3D scanner, as is shown in Figure 3.2. The general procedure of this method includes 3D scanning, 3D modeling and 3D printing [5]. The whole process is time-consuming. What's more, it doesn't consider the deformation of the soft tissue during the measurement. Only by deforming the soft tissue on the foot can the measurement best simulate the condition when a human is standing on the orthoses. On top of these, the price of these devices can easily exceed 10,000 RMB.

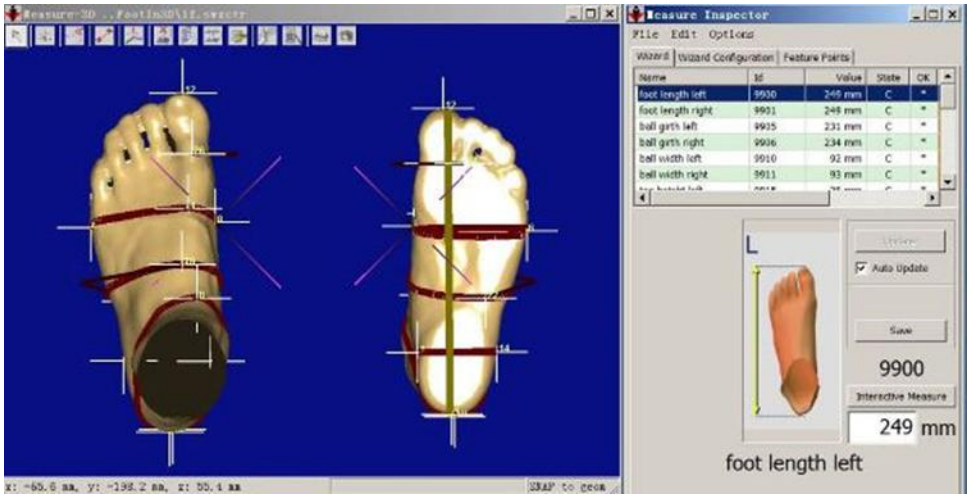


Figure 3.2 The results and data gathered by a 3D scanner

In brief, flat foot is a common disease which can be treated with foot orthoses. Although there already exists 3D scanning method to measure and build orthoses, it still:

- Takes a lot of time to complete the 3D scanning, 3D modeling, and 3D printing process.
- Costs much money to buy the devices
- Doesn't consider soft tissue deformation during the measurement.

IV. Needs

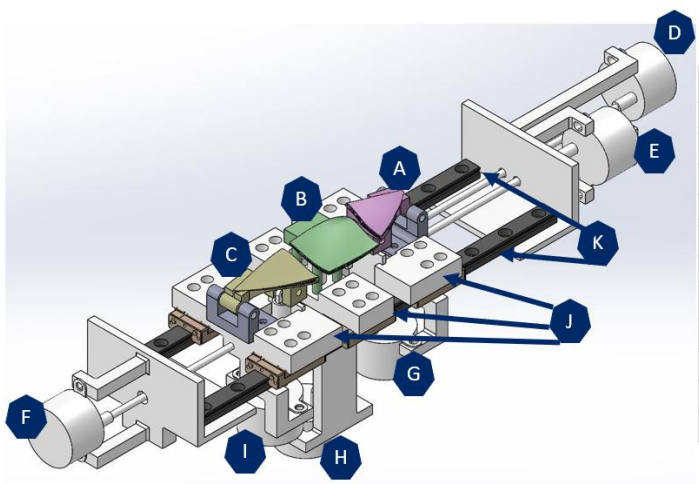
To solve the problems in foot arch orthoses technology, the new model needs to:

1. Measure the foot arch while applying pressure
2. Show the pressure explicitly
3. Fit the foot arch of the patient automatically and appropriately
4. Be controlled by motors
5. Be manually adjustable to fit customized requirements
6. Reproduce the surface with given parameters

V. Solution: SmartArch

A new foot orthoses measuring system is designed with customable parts movement and a built-in algorithm to let the parts fit the foot arch automatically. It provides:

- **Customizable pressure on the foot arch at different parts.**
- **Quick and easy automatic fitting of a person's foot arch.**
- **Showing of the data of the measurement for later reference**
- **Reconstruction function where data can be inputted to give a surface of a foot arch.**



ITEM	DESCRIPTION
A	Front segment
B	Mid segment
C	Rear segment
D	Step motor controlling X_{fore}
E	Step motor controlling X_{mid}
F	Step motor controlling X_{rear}
G	Step motor controlling $\theta_{\text{z,fore}}$
H	Step motor controlling Y_{mid}
I	Step motor controlling $\theta_{\text{z,rear}}$
J	Supporting subassemblies
K	Linear guide

Figure 5.1: SmartArch shown in a 3D model

The proposed new foot orthoses measuring system is shown schematically in Figure 5.1. The system has three main structural segments, namely the front, mid and rear segments. These segments serve as the parts that fit the foot arc. They are designed to have the shape of a person's foot arch based on data from 88 subjects [6].

Using our built-in interactive system in SmartArch, the user can freely command SmartArch to perform different actions. It can fit the user's foot arch automatically, as

well as be adjusted manually using an IR remote control. It can also show the result of this measurement to the user. Besides these, the customer can input some data, and SmartArch will move its segments to give a surface according to the data. All of these algorithms are built inside the Arduino board, which controls the stepper motors according to the input.

The user can have quick and easy access to a measurement of his own foot arch, as well as have control over the final shape of the measurement using SmartArch in the ways mentioned above. Moreover, the user can have the measurement taken when pressure is applied on the foot, as SmartArch needs a foot to be stepped on it to work. This proves to be SmartArch's biggest edge over its competitors, as the most prevalent ways do not take measurement with pressure applied.

VI. Objectives

- **Objective 1: Design a foot orthoses mold that can apply pressure to the foot arch**

Design a system that prevails over the 3D scanning solution by applying pressure to the foot arch. This serves as the major advantage of this design over other solutions.

- **Objective 2: Build a prototype demonstrating its functions**

To let it be used and tested in real life, the machine must be made out concretely.

- **Objective 3: Let the prototype be controlled easily**

The prototype should first fit the foot arch automatically, and then manual adjustments should be applied. These should be controlled electronically. The prototype should also output the data needed to reconstruct the surface. In this way, the surface can be acquired again by inputting these data.

- **Objective 4: Test and evaluate the prototype**

The prototype should be ensured to work properly.

- **Objective 5: Reporting**

The ways to fabricate the machine should be documented so that it can be re-created by following the same steps.

VII. Tasks

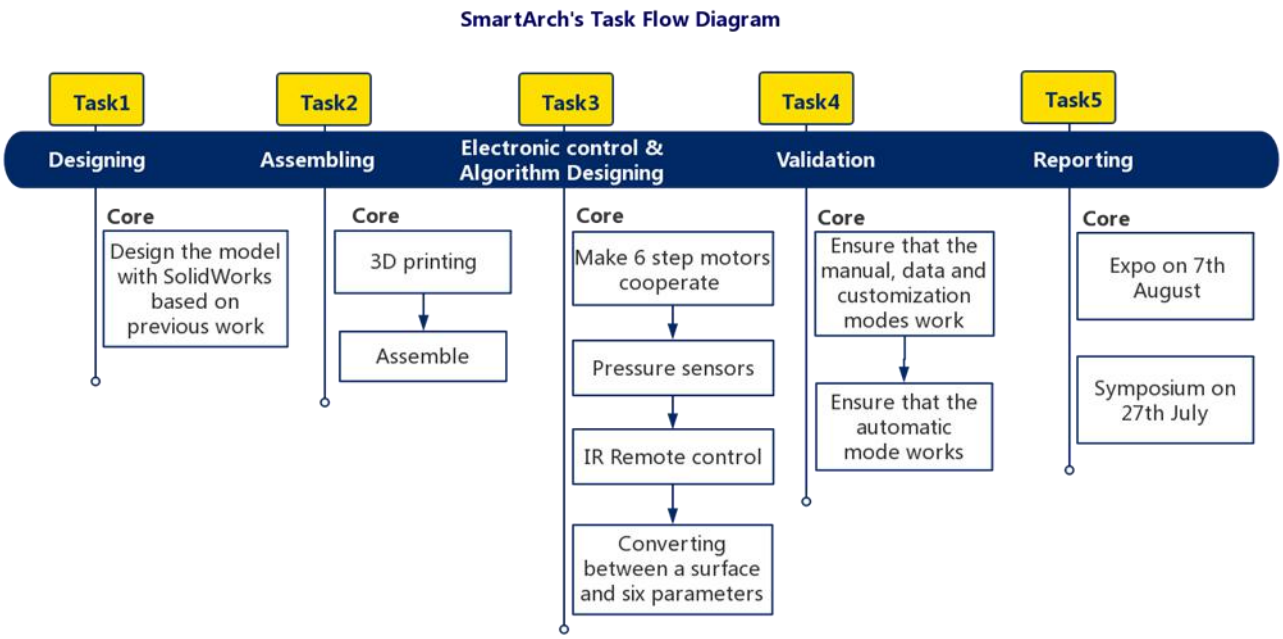


Figure 7.1 SmartArch's task flow diagram

- **Task 1: Use SolidWorks to design a 3D model of the prototype based on previous foot arch researches**

The objective of task 1 is to build a 3D model of every part of the foot arch orthoses mold and find out the way to move the system. The detailed tasks included: designing surfaces that can best fit the foot arch of most people, designing a movable structure that hold the surfaces.

Although the foot arches of different people vary, they can be generally presented by some certain shapes of surfaces. Based on former researches [6], a combination of

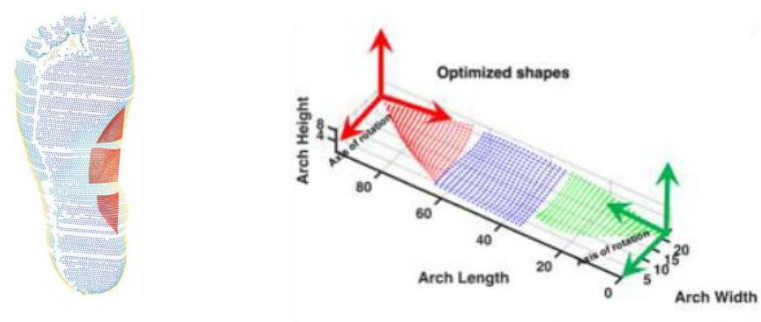


Figure 7.2 Three surfaces that can generally present people's foot arches [3]

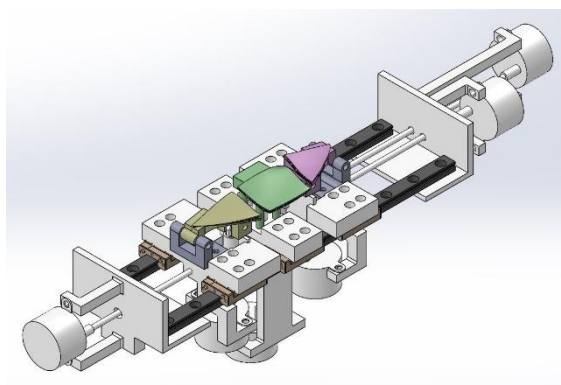


Figure 7.3 The 3D view of the whole system

three surfaces is built to shape a wide range of foot arches (Figure 7.1).

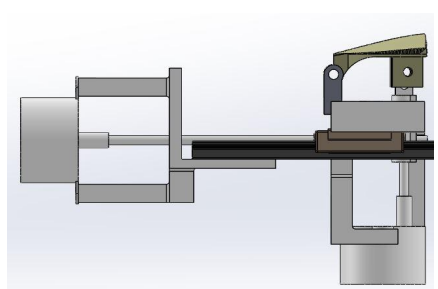


Figure 7.4: The stepper motors and lead screws controlling the rear segment

A movable system was designed using SolidWorks to change the positions of the three surfaces to fit different foot arches with different arch lengths and depths. As is

shown in Figure 7.2, the system has three bases fixed on six sliding blocks locked on two parallel rails. Each block is connected to a stepper motor with a lead screw so that it can be moved horizontally. The surfaces on both sides are fixed on axes respectively, and have the other end connected to stepper motors with lead screws on the bottom. In this way, the angle formed by them and the horizontal surface can also be adjusted by rotating them around the axis. This is shown more clearly in Figure 7.3, which features two stepper motors, each connected with a lead screw. Finally, the middle surface also moves up and down with the control of a stepper motor and a lead screw. In all, the system has 6 degrees of freedom, all controlled by stepper motors and lead screws, which can move the surfaces more precisely and hold them more steadily.

After the completion of Task 1:

- A 3D model of surfaces that best present humans' foot arches was built.
- A movable structure holding all actuators and surfaces with 6 degrees of freedom was designed.

- **Task 2: 3D-print the parts and assemble the prototype**

After having generally designed a prototype using SolidWorks, we put our design into manufacturing. Initially, we decided to use 3D printing to print out the main structure of our measurement device, namely the three segments that directly contact the feet of the patients. We also purchased and installed a pair of linear guides, where the supporting subassemblies can move back and forth to fit foot arches of a variety of sizes.

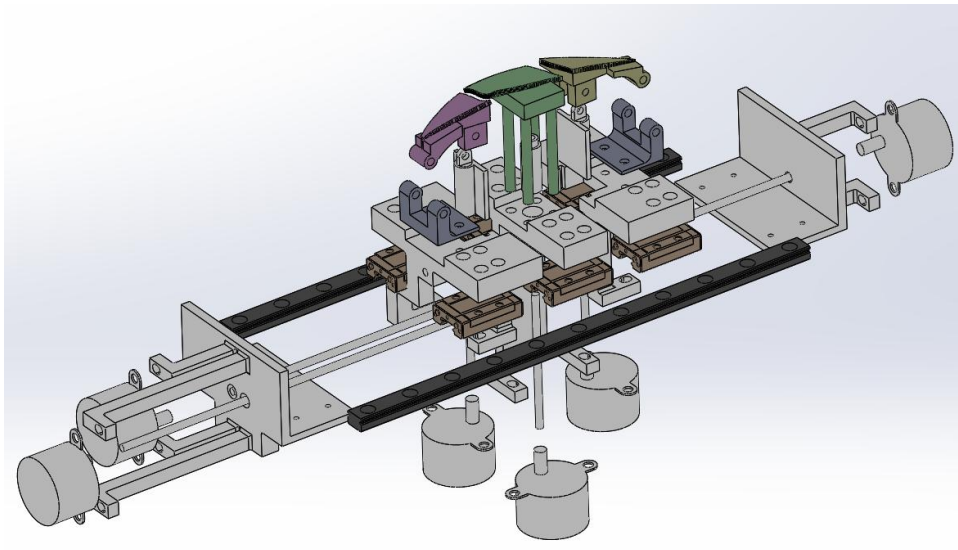


Figure 7.5 The parts of SmartArch in exploded view

Motor	Parameter	Pressure sensor
1	X_{rear}	P1
2	X_{fore}	P6
3	X_{mid}	P3, P4
4	$\theta_{\text{z,rear}}$	P2
5	Y_{mid}	P3, P4
6	$\theta_{\text{z,fore}}$	P5

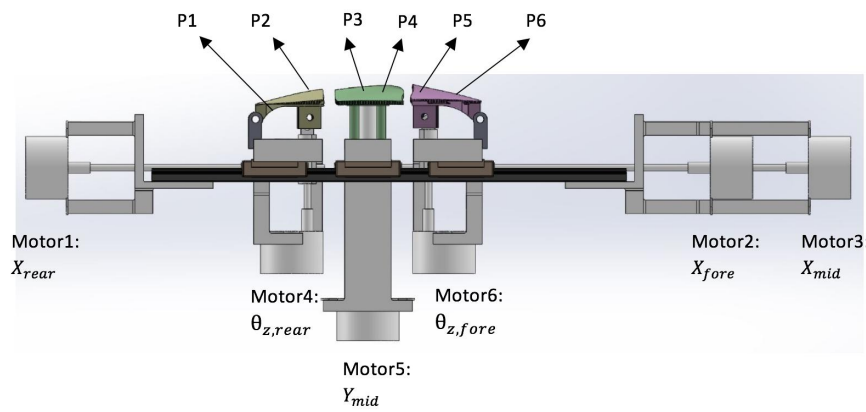


Figure 7.6: The numbers of the pressure sensors and motors, and the degrees of freedom the motors control

The connections between motors and the segments are important components of our prototype because they affect the speed of our segment significantly. We initially applied ordinary lead screws to conduct the movement of the motor, but they turned out to be so slow that the change in position happening on the segments is unnoticeable. We then applied high-speed lead screw to help accelerate the motion, and the change was satisfactory.

After the completion of Task 2:

- SmartArch had its the preliminary shape and was ready for the electronic control system to be installed.

- **Task 3: Designing and coding the electronic control system to provide better control over the prototype**

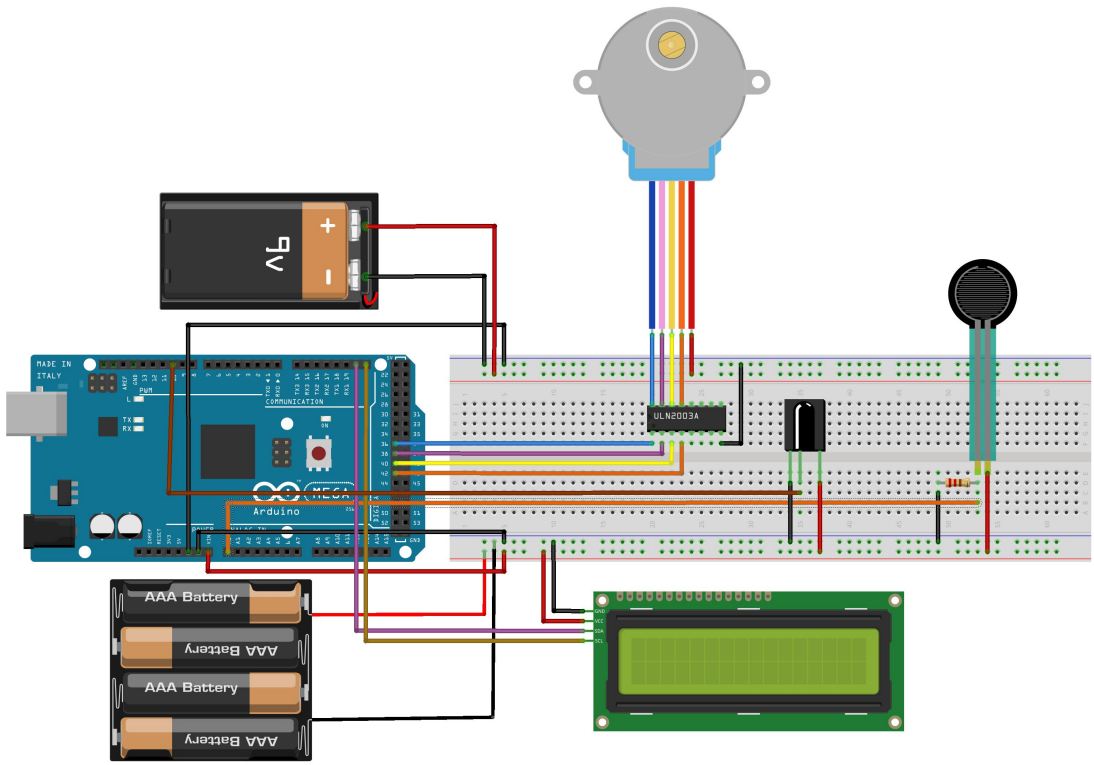


Figure 7.7 The circuit diagram of SmartArch

In the same Arduino circuit, we use two different voltages: 5V and 12V. 12V is used to power the six stepper motors, while 5V is used to power the Arduino Mega Board, the LCD screen, the IR remote receiver and six pressure sensors. In addition, every pressure sensor is attached to a pressure sensor converter, which can convert the electric signal into analog signal.

We chose the combination of stepper motors and lead screws because its accuracy is high: the speed of the stepper motors can be adjusted and the total steps that it rotates can be counted and recorded. In our project, we use this combination to control the magnitude of the six parameters: X_{fore},X_{mid}, X_{rear}, $\theta_{\text{z,fore}}$, Y_{mid},$\theta_{\text{z,rear}}$.

What's more, pressure sensors are used to reflect the deformation of soft tissue of the foot arch. In order to read the pressure from the analog signal, a small experiment was carried out. Through linear fit, we derived the following equation:

$$F = 37909 \times \frac{1}{A} - 416.3,$$

where A is the analog signal received by the Arduino Mega board, and F is the force applied. The complete procedures for the experiment are in Appendix A.

Since our device deals with many input, output and different modes, a small interactable system was written in Arduino. Input and output are realized by the IR remote control and the LCD screen respectively.

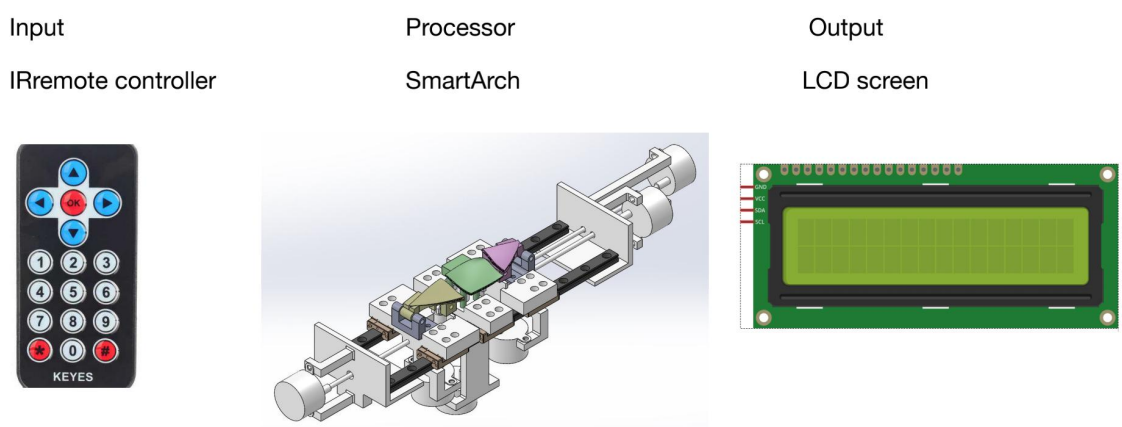


Figure 7.8 SmartArch's data processing processes

The basic mode is the menu mode, in which you can choose the following modes for the device.

1. Automatic mode:

SmartArch will fit the foot arch automatically. This is realized by the cooperation of stepper motors and pressure sensors. The basic idea is that stepper motors will move the segments forward orderly. And they will keep rotating until the pressure at corresponding point is larger than a certain numerical value. In our case, the criterium is F > 100g.

2. Manual mode:

This mode allows the user to use a remote controller to adjust six stepper motors. The user can choose which motor to move according to the instructions on the screen. Then if the button "<" pressed, the selected motor will rotate forward until the button "<" is pressed again. The button ">" works in a similar way. To increase the accuracy, the stepper motors will only rotate 0.1 step every time. Therefore, the maximum uncertainty of our device is

$$0.1 \times 2\text{mm} = 0.2\text{mm}.$$

After the customer is satisfied, he can press the button "OK" to return to the menu.

3. Data mode:

The magnitude of six parameters (X_{fore},X_{mid}, X_{rear}, $\theta_{\text{z,fore}}$, Y_{mid},$\theta_{\text{z,rear}}$) and pressure at 5 points (P1, P2, P3, P4, P5) will be shown on the LCD screen (see Figure 7.7). As is mentioned before, the total steps the motor rotates will be recorded. Since the pitch of the lead screw is 2mm, the displacement distance can be calculated as

$$d = 2\text{mm} \times n,$$

where d is the displacement distance, and n is the total steps the motor rotates, which can be negative or decimals. After the customer is satisfied, he can press the button "OK" to

return to the menu.

4. Customization mode:

The user can input six parameters and SmartArch will return a surface that represents the foot arch. When inputting numbers through remote controller, the input is displayed on the LCD screen real-time. And since there is no ready-made code in Arduino, we wrote the corresponding function by ourselves. After inputting all parameters and pressing the button “OK”, the six stepper motors will rotate orderly until they reach the designated places. When inputting the numbers, the button “#” represents negative sign and the button “*” represents decimal point. In the case of mistype, the button “<” can be pressed to delete the input.

After the completion of Task 3:

- SmartArch could be controlled electronically.
- SmartArch has its four modes. Theoretically, it could function.

- **Task 4: Validation**

Though we designed SmartArch well beforehand, several problems occurred when we tested it. One difficult part in our process is the realization of electronic control. Many potential errors happened, and we figured out the reason and fixed it. For instance, sometimes Dupont wires lost contact with Arduino board or the motors, and there was no movement after the input was given by the remote control. To improve, we later rearranged the electronic wires in a neat and clear way to avoid interference between Dupont wires.

After fixing the minor problems, we found a volunteer to put his foot onto SmartArch and use it, as is shown in Figure 7.9. When we lifted the parts up, his foot clearly felt the pressure, which means that SmartArch can function successfully. However, SmartArch did not stop when much pressure was applied on it, and that part was put to later tests to make sure it works properly. After that, we also tested the customization mode and the data mode. These turned out to work perfectly. SmartArch could move its parts to give a shape of a foot arch, and after each measurement, it could show the respective data on the screen.

When the measurement task is completed, we can record the position of the foot arch and we can thus use these data as reference to help us construct the orthoses for the foot arch.

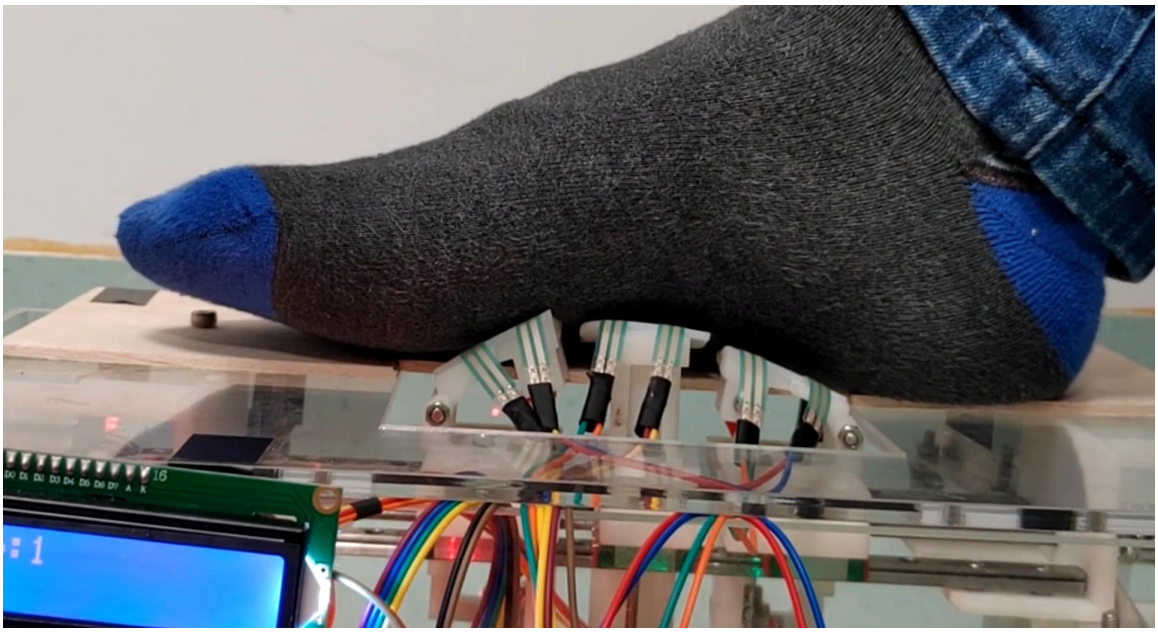


Figure 7.9 The validation process

After the completion of Task 4:

- SmartArch was ensured to function properly.

Task 5: Reporting

To advertise our product and gain better tech-communication effect, we have written reports, made presentations and designed posters to illustrate its advantages and potential development. During the symposium on 27th July, an oral presentation was done, and pictures and videos were provided for viewers to attain further knowledge about how to use our devices more easily. During the expo on 7th August, live demonstrations were made to better advertise our product to a wider audience.

After the completion of Task 5:

- People know more about SmartArch and what it can do.

VIII. Discussion

By finishing this project, we have achieved in giving SmartArch distinct advantage over its alternatives. We have successfully manufactured a machine that applies pressure on the foot arch to measure the shape of the foot in a suitable condition.

Through working on this project, we have learnt that engineering is not just about turning a design draft into a prototype. The designing process itself is also very painstaking. Also, after the construction of the prototype, a huge number of problems will emerge that couldn't be solved in draft stage. Thus, we have developed the ability to find the cause and solve it whenever something goes wrong. Besides these, there are matters to be considered beyond designing and manufacturing, such as costs and using experience. Only after the prototype is fully tested and validated in all respects, can we claim that we have achieved our goal.

However, the design still has some room for improvement. As a human foot can't be expressed in three parts, it may be better if we used more parts to fit the foot arch. Also, the design requires the patient to set foot on it, which may cause the patient's disgust, or even cause skin disease to spread. It would be better if it was manufactured with a germ-free surface, and sterilized after each use.

IX. Conclusion

In this project, we have developed a foot orthoses mold that can apply pressure to the foot arch. To control it, we have also developed an easy-to-use electronic control system with various modes. The patient can just put the foot on the mold and control it with a remote control without any mechanical knowledge. By applying pressure to the foot, the measure is more accurate than its alternatives which can only measure the shape of the foot in its neutral position. Accurate measuring is our major advantage. Also, our design costs less money and most of the material in the design is not difficult to get, which means that it can be used widely. We hope it can be used more widely so that more flat foot patients as well as pediatrists can enjoy the cheap yet effective design.

X. Schedule

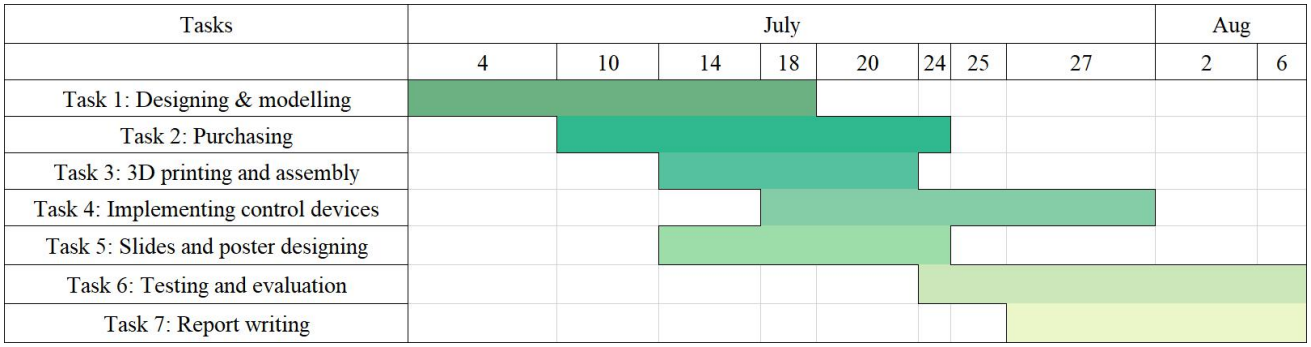


Figure 10.1. Gantt chart

XI. Bill of Materials

Quantity	Part Description	Purchased From	Part Number	Price (yuan each)
1	Acrylic box	Taobao Vendor[1]	300*200*120mm	150.00
1	Arduino Board	Taobao Vendor[2]	Mega Board	308.00
1	Button cell	Taobao Vendor[3]	CR2025 3V	2.38
6	Stepper motor	Taobao Vendor[4]	29BYJ-48-12V	5.38
5	Pressure sensor probes	Taobao Vendor[5]	602, 20kg	20.00
6	Stepper motor driver board	Taobao Vendor[6]	UL2003	1.80
20	Screws & nuts	Shanghai Jiao Tong University	M3, M4	0.00
3+3	Dupont wires	Taobao Vendor[7]	40p, half bronze, 21cm, male to male + male to female	3.50
1	Ruler sticker	Taobao Vendor[8]	20cm*1cm, white	11.80
1	LI-PO Battery	Taobao Vendor[9]	4.8V	32.21
6	Sliding block	Taobao Vendor[10]	MGN7C	23.00
1	3D-printted workpieces	Taobao Vendor[11]	photopolymer	195.00
1	Screws & nuts	Taobao Vendor[12]	M2, M3	5.03
2	Sliding rail	Taobao Vendor[13]	MGN7-100mm	26.00
6	Couplings	Taobao Vendor[14]	3*5mm	5.50
6	Ball bearings	Taobao Vendor[15]	MR63ZZ	2.60
1	LCD panel	Taobao Vendor[16]	L2C 1602A	7.36
1	Remote control & receiver	Taobao Vendor[16]	IR remote	10.00
3	Signal converter	Taobao Vendor[5]		15.00
4	Aluminum bar	Taobao Vendor[17]	10*10*200 mm angle	2

			aluminum	
1	Heel cup	Taobao Vendor[18]	Blue, male	15.00
1	Lead screws	Taobao Vendor[19]	3mm*120mm	16.00
2	Lead screws	Taobao Vendor[20]	3mm*120mm with sliding blocks	17.00
	TOTAL			**1237.84**

Table 11.1. Budget table

1. https://item.taobao.com/item.htm?spm=a1z10.1-c.w4004-5508338049.3.4a2d6355em5M1z&id=595673883653
2. https://detail.tmall.com/item.htm?spm=a1z10.5-b.w4011-2740029678.69.6a072765H3y62X&id=38816270178&rn=6174c438d246dd8d755f8341e874d16f&abbucket=6
3. https://detail.tmall.com/item.htm?spm=a1z10.3-b-s.w4011-14674067888.34.281939a2qSNtX4&id=40415813883&rn=1ba534a44c1fd6daee9858b2c1862fa0&abbucket=6
4. https://detail.tmall.com/item.htm?spm=a1z10.3-b-s.w4011-16538328900.46.6e1c3d6cbkWpVK&id=41320614619&rn=d93a0bcad3c4f02a5d6e6a3a138b31d7&abbucket=6
5. https://item.taobao.com/item.htm?spm=a1z10.3-c.w4002-18792858412.37.51d5699aybASqk&id=596913817159
6. https://item.taobao.com/item.htm?spm=a1z10.3-c-s.w4002-16248799899.11.1a1d546e5rOTj7&id=36134486250
7. https://item.taobao.com/item.htm?spm=a1z10.3-c.w4002-16408964706.9.8a945732P8eUnr&id=585008907113
8. https://item.taobao.com/item.htm?spm=a1z10.5-c.w4002- 15775900266.14.457d6a53FU9uNy&id=592748024743
9. https://shop284714700.taobao.com/?spm=a230r.7195193.1997079397.4.6dbf1980sYMYjM
10. https://detail.tmall.com/item.htm?spm=a1z10.5-b.w4011-16008545442.63.15fd5f76IDJAHG&id=582171993899&rn=f0f12b6424e1a7feec6f6da69ed56057&abbucket=6
11. https://detail.tmall.com/item.htm?spm=a1z10.5-b.w4011-14920583976.15.573131d8Yi0hZg&id=43901666034&rn=f545ea05e8aab05c52d490845282d4e6&abbucket=6
12. https://jinchaowj.tmall.com/index.htm?spm=a1z10.3-b.w5002-16847363435.2.576b41b82CxNXq
13. https://raumu.tmall.com/shop/view_shop.htm?spm=a230r.7195193.1997079397.2.7216ca88cXatew
14. https://shop191038982.taobao.com/?spm=a230r.7195193.1997079397.2.a79c1f7c1AaMBp
15. https://shop143079358.taobao.com/?spm=a230r.7195193.1997079397.2.1b792a47XUpuC1
16. https://market.m.taobao.com/app/tb-source-app/wz111/pages/impression?pathInfo=shop/impression&userId=1655664677&shopId=104368608&wh_weex=true&pageId=0&alisite=true
17. https://shop62268257.taobao.com/?spm=a230r.7195193.1997079397.2.2ec917a6Qp1dLb
18. https://hezujian.tmall.com/shop/view_shop.htm?spm=a230r.7195193.1997079397.13.280b74da42uILi
19. https://shop64800317.taobao.com/?spm=a230r.7195193.1997079397.2.161f4b0e3W2Ff0
20. https://shop486696305.taobao.com/?spm=a230r.7195193.1997079397.2.65ad350eINwMvq

XII. Key Personnel

	Personnel	Principal roles
Key personnel	Yiding Chang	Group leader & head of technical communications
	Zhongqian Duan	Head of electrical devices and controlling
	Chen Huang	Head of structure and modelling
	Haotian Xie	Head of purchasing
	Lang Shen	Other support
Other personnel	Prof. Shane Johnson	Provide structural advice
	Prof. Michele Campbell	Provide technical communication advice
	Siwei Ye	Provide structural advice
	Haichao Yang	Provide structural advice
	Tian Gao	Provide technical communication advice
	Yating Liu	Provide technical communication advice
	Otto Zhang	Provide structural advice

Table 12.1. Key personnel

XIII. References

[1] Ezema, C. I., U. O. Abaraogu, and G. O. Okafor. "Flat foot and associated factors among primary school children: A cross-sectional study." Hong Kong Physiotherapy Journal 32.1(2014):13-20.

[2] Blog.sina.com.cn. (2019). Why is walking with flat feet tiring?_bskx_Sina Blog. [online] Available at: http://blog.sina.com.cn/s/blog_7a1f8b020102w049.html [Accessed 4 Aug. 2019].

[3] Rensheng2.com. (2019). How to determine your foot arch type?_ Rensheng2. [online] Available at: http://www.rensheng2.com/1810000/1803339.shtml [Accessed 4 Aug. 2019].

[4] A. Saxena and J. Haddad, The effect of foot orthoses on patellofemoral pain syndrome. Journal of the American Podiatric Medical Association, (2003). 93(4): p. 264-271.

[5] Printers, D., Printers, I., Materials, F., Powder, M., Powder, P., Service, 3. and us, C. (2019). What is 3D printing? How does a 3D printer work? Learn 3D printing. [online] 3D Printing. Available at: https://3dprinting.com/what-is-3d-printing/ [Accessed 4 Aug. 2019].

[6] Qaiser, Zeeshan, and Faraz, Ahmad, and Johnson, Shane. “Feasibility of a novel 3D ergonomic measurement system for custom foot orthoses”.

Appendix A: The Data of the Linear Fit for the Pressure Sensor

There exists a linear relationship between the force applied on the pressure sensor and the reciprocal of the analog signal received by the Arduino Mega board. An experiment was conducted to find out the correct formula. We first measured several sets of data for the pressure applied and the analog signal. The analog signal received is denoted as A below. It ranges from 100 to 0.

F [g]	A
100	80
200	60
300	50
400	46
500	42
600	37
700	34
800	31
900	29
1000	27

Table A.1 The data for the pressure applied and the analog signal

Based on the data in Table A.1, we applied linear fit to F vs. $\frac{1}{A}$, and the result is shown in Figure A.1, from which we get Equation 1. Thus, we get the formulae needed in this part.

$$F = 37909 \times \frac{1}{A} - 416.3, \text{ (Eq. 1)}$$

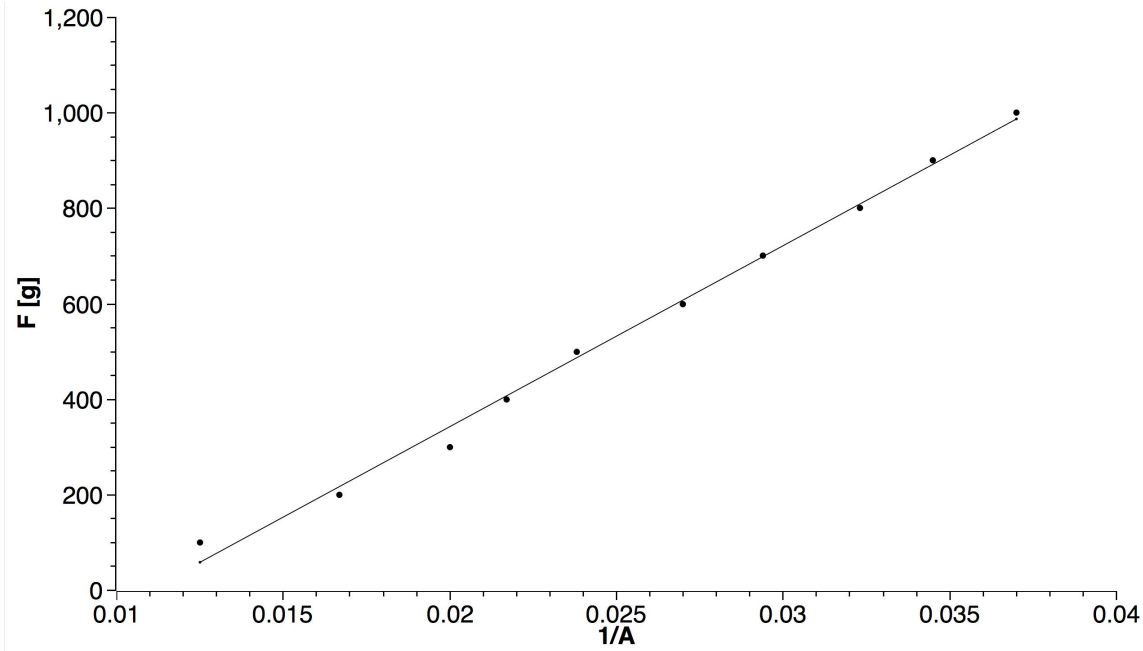


Figure A.1 The linear fit of F vs. $\frac{1}{A}$.

Appendix B: The Corresponding Signal of the IR Remote Control's Buttons

Table B.1 contains information about what signal will be emitted when the respective buttons are pressed on the IR remote control. Though the same information is provided at the end of the full codes in Appendix C, providing a list here can provide the reader with the necessary information for reading the codes.

Button	Signal
1	16753245
2	16736925
3	16769565
4	16720605
5	16712445
6	16761405
7	16769055
8	16754775
9	16748655
*	16738455
0	16750695
#	16756815
ok	16726215
↑	16718055
↓	16730805
←	16716015
→	16734885

Table B.1 The corresponding signal of the IR remote control's buttons.

Appendix C: Full Codes

```
#include <Stepper.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <IRremote.h>
#define STEPS 2038
int mode = 5;
int initialized = 0;
//1 for manul,2 for auto,3 for Input
//4 for data, 5 for initialize, 0 for menu

//F =  37909/A - 416.3

//-----------------step motor----------------
Stepper stepper0(STEPS, 22, 26, 24, 28);//Xfore
Stepper stepper1(STEPS, 30, 34, 32, 36);//Xrear
Stepper stepper2(STEPS, 38, 42, 40, 44);//Xmid
//Stepper stepper3(STEPS, 46, 50, 48, 52);//Ymid
Stepper stepper3(STEPS, 52, 48, 50, 46);//Ymid
//Stepper stepper4(STEPS, 23, 27, 25, 29);//Theta1
Stepper stepper4(STEPS, 29, 25, 27, 23);//Theta1
//Stepper stepper5(STEPS, 31, 35, 33, 37);//Theta2
Stepper stepper5(STEPS, 37, 33, 35, 31);//Theta2
double Step2Distance = 2; //1 step -- 2mm
double stepCount[6];
double targetCount[6];
double stepReadings[6];
int MaxSpeed = 15; //15 rpm
//-----------------Pressure sensor------- ------
int sensor[6] = {A0, A1, A5, A4, A3, A2};
//Xfore, Theta1, Xrear, Theta2, Ymid , Xmid

int sensorRead[6];
int newdata[6];
double PressureReadings[6];
//-----------------IRremote-------------------
int RECV_PIN = 10;
IRrecv irrecv(RECV_PIN);
decode_results results;
//-----------------I2C-------------------
LiquidCrystal_I2C lcd(0x27, 20, 4);
int menu_initialized = 0;
//----------------Function---------------------
void Initialize();
void Menu();
void Menu_assit();
int Menu_assit_delay();
void Data();
int Data_assist();
void Input();
void Input_assist();
double String2Double();
int FindDecimalPoint();
void Input_control();
Stepper StepperIndex();
void manual_control();
```

```
void manual_control2();
int manual_control_delay();
void manual_control_assist();
void auto_control();
void auto_control_assist();
void SensorRead();
void PrintStep();
//-----------------setup----------------------
void setup()
{
  Serial.begin(9600);
  irrecv.enableIRIn();
  Serial.println("Enabled IRin");
  lcd.init();
  lcd.backlight();
  Serial.println("Enabled lcd");

}
void loop()
{
  switch (mode)
  {
    case 0:
      {
        Menu();
        break;
      }
    case 1:
      {
        manual_control2();
        break;
      }
    case 2:
      {
        auto_control();
        break;
      }
    case 3:
      {
        Input();
        break;
      }
    case 4:
      {
              Data();
              break;
            }
          case 5:
            {
              if (initialized == 0)
              {
                Initialize();
                break;
              }
            }

        }
      }
      void Initialize()
```

```
    {
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Initializing");
      lcd.setCursor(7, 1);
      lcd.print("SmartArch");
      delay(1000);
      manual_control2();
      for (int i = 0; i < 6; i++)
      {
        stepCount[i] = 0;
        initialized = 1;
      }
    }

    void Menu()
    {
      Serial.print("Mode:"); Serial.println(0);
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Welcome to");
      lcd.setCursor(6, 1);
      lcd.print("SmartArch!");
      delay(1000);
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("1 Manul,2 Auto");
      lcd.setCursor(0, 1);
      lcd.print("3 Input,4 Data");
      irrecv.resume();
      while (!(irrecv.decode(&results)))
      {
        delay(100);
      }
      lcd.clear();
      Serial.print("IRremote:");
      Serial.println(results.value);
      switch (results.value)
      {
        case 16753245:
          {
            lcd.setCursor(0, 0);
            lcd.print("Manul mode?");
            Menu_assit(1);
            return;
          }
        case 16736925:
          {
            lcd.setCursor(0, 0);
            lcd.print("Auto mode?");
            Menu_assit(2);
            return;
          }
        case 16769565:
          {
            lcd.setCursor(0, 0);
            lcd.print("Input mode?");
            Menu_assit(3);
            return;
```

```
            }
          case 16720605:
            {
              lcd.setCursor(0, 0);
              lcd.print("Data mode?");
              Menu_assit(4);
              return;
            }
        }
      }

      void Menu_assit(int i)
      {
        lcd.setCursor(0, 1);
        lcd.print("OK or Left arrow");
        irrecv.resume();
        if (Menu_assit_delay() == 1)
        {
          if (results.value == 16726215)
          {
            mode = i;
            Serial.print("Mode:"); Serial.println(i);
            lcd.clear();
            lcd.setCursor(0, 0);
            lcd.print("Mode:");
            lcd.setCursor(5, 0);
            lcd.print(mode);
            delay(1000);
          }
          else if (results.value == 16716015)
          {
            mode = 0;
          }
        }
      }

      int Menu_assit_delay()
      {
        while (!(irrecv.decode(&results)))
        {
          delay(100);
        }
        Serial.println(results.value);
        if (results.value == 16726215 || results.value == 16716015)
        {
          Serial.println("Return1");
          return 1;
        }
        else
        {
          irrecv.resume();
          Serial.println("Delay");
          Menu_assit_delay();
        }
      }

      void Data()
      {
        for (int i = 0; i < 6; i++)
```

```
        {
          stepReadings[i] = stepCount[i] * Step2Distance;
        }
        for (int i = 0; i < 6; i++)
        {
          //double R = newdata[i] * 0.187 - 3.217;
          PressureReadings[i] = 37909 / newdata[i] - 416.3;
          //F =  37909/A - 416.3
        }
        while (true)
        {
          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("P1.1:");
          lcd.setCursor(5, 0);
          lcd.print(PressureReadings[0]);
          lcd.setCursor(0, 1);
          lcd.print("P1.2:");
          lcd.setCursor(5, 1);
          lcd.print(PressureReadings[1]);
          if (Data_assist() == 1)
          {
            mode = 0;
            return;
          }

          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("P2.1:");
          lcd.setCursor(5, 0);
          lcd.print(PressureReadings[2]);
          lcd.setCursor(0, 1);
          lcd.print("P2.2:");
          lcd.setCursor(5, 1);
          lcd.print(PressureReadings[3]);
          if (Data_assist() == 1)
          {
            mode = 0;
            return;
          }

          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("P3.1:");
          lcd.setCursor(5, 0);
          lcd.print(PressureReadings[4]);
          lcd.setCursor(0, 1);
          lcd.print("P3.2:");
          lcd.setCursor(5, 1);
          lcd.print(PressureReadings[5]);
          if (Data_assist() == 1)
          {
            mode = 0;
            return;
          }


          lcd.clear();
          lcd.setCursor(0, 0);
```

```
          lcd.print("Xfore:");
          lcd.setCursor(6, 0);
          lcd.print(stepReadings[0]);
          lcd.setCursor(0, 1);
          lcd.print("Xrear:");
          lcd.setCursor(6, 1);
          lcd.print(stepReadings[1]);
          if (Data_assist() == 1)
          {
            mode = 0;
            return;
          }


          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Xmid:");
          lcd.setCursor(5, 0);
          lcd.print(stepReadings[2]);
          lcd.setCursor(0, 1);
          lcd.print("Ymid:");
          lcd.setCursor(5, 1);
          lcd.print(stepReadings[3]);
          if (Data_assist() == 1)
          {
            mode = 0;
            return;
          }

          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Theta1:");
          lcd.setCursor(7, 0);
          lcd.print(stepReadings[4]);
          lcd.setCursor(0, 1);
          lcd.print("Theta2:");
          lcd.setCursor(7, 1);
          lcd.print(stepReadings[5]);
          if (Data_assist() == 1)
          {
            mode = 0;
            return;
          }
        }
      }

      int Data_assist()
      {
        int currentDelay = 2000;
        int interval = 100;
        irrecv.resume();
        while (currentDelay > 0)
        {
          delay(interval);
          currentDelay -= interval;
          if (irrecv.decode(&results))
          {
            if (results.value == 16726215)
            {
```

```
                return 1;
              }
            }
          }
          return 0;
        }

        void Input()
        {
          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Xfore:");
          lcd.setCursor(0, 1);
          lcd.print("Xrear:");
          Input_assist(6, 0, 0);
          Input_assist(6, 1, 1);


          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Xmid:");
          lcd.setCursor(0, 1);
          lcd.print("Ymid:");
          Input_assist(5, 0, 2);
          Input_assist(5, 1, 3);

          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Theta1:");
          lcd.setCursor(0, 1);
          lcd.print("Theta2:");
          Input_assist(7, 0, 4);
          Input_assist(7, 1, 5);

          Input_control();

          mode = 0;

        }
        void Input_control()
        {
          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Input Mode");
          lcd.setCursor(0, 1);
          lcd.print("controlling...");
          for (int i = 0; i < 6; i++)
          {
            Stepper CurrentStepper = StepperIndex(i);
            CurrentStepper.setSpeed(MaxSpeed);
            CurrentStepper.step((targetCount[i] - stepCount[i])*STEPS);
            stepCount[i] = targetCount[i];
          }
        }
        Stepper StepperIndex(int i)
        {
          switch (i)
          {
            case 0:
```

```
            {
              return stepper0;
            }
          case 1:
            {
              return stepper1;
            }
          case 2:
            {
              return stepper2;
            }
          case 3:
            {
              return stepper3;
            }
          case 4:
            {
              return stepper4;
            }
          case 5:
            {
              return stepper5;
            }
        }
      }


      void Input_assist(int CursorX, int CursorY, int i)
      {
        String result;
        int isFirst = 1;
        while (true)
        {
          irrecv.resume();
          while (!irrecv.decode(&results))
          {
            delay(100);
          }
          switch (results.value)
          {
            case 16756815:
              {
                if (isFirst == 1)
                {
                  result += '-';
                  lcd.setCursor(CursorX, CursorY);
                  lcd.print(result);
                  isFirst = 0;
                  break;
                }
              }
            case 16753245:
              {
                result += '1';
                lcd.setCursor(CursorX, CursorY);
                lcd.print(result);
                break;
              }
            case 16736925:
```

```
            {
              result += '2';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }
          case 16769565:
            {
              result += '3';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }

          case 16720605:
            {
              result += '4';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }
          case 16712445:
            {
              result += '5';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }
          case 16761405:
            {
              result += '6';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }

          case 16769055:
            {
              result += '7';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }
          case 16754775:
            {
              result += '8';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }
          case 16748655:
            {
              result += '9';
              lcd.setCursor(CursorX, CursorY);
              lcd.print(result);
              break;
            }
          case 16750695:
            {
```

```
                result += '0';
                lcd.setCursor(CursorX, CursorY);
                lcd.print(result);
                break;
              }
            case 16716015:
              {
                result = result.substring(0, result.length() - 1);
                lcd.setCursor(CursorX, CursorY);
                lcd.print(result);
                lcd.setCursor(CursorX + result.length(), CursorY);
                lcd.print(' ');
                break;
              }
            case 16738455:
              {
                result += '.';
                lcd.setCursor(CursorX, CursorY);
                lcd.print(result);
                break;
              }
            case 16726215:
              {
                targetCount[i] = String2Double(result);
                Serial.println(result);
                Serial.print("targetCount:");
                Serial.println(targetCount[i]);
                return;
              }
          }
        }
      }

      double String2Double(String input)
      {
        double output;
        int k = FindDecimalPoint(input);
        int len = input.length();
        if (input[0] != '-')
        {
          if (k == 0)
          {
            for (int i = 0; i < len; i++)
            {
              int x  = input[i] - '0';
              output += x * pow(10, len - i - 1);
            }
          }
          else
          {
            for (int i = 0; i < k; i++)
            {
              int x  = input[i] - '0';
              output += x * pow(10, k - i - 1);
            }
            for (int i = k + 1; i < len; i++)
            {
              int x = input[i] - '0';
              output += x * pow(10, k - i);
```

```
            }
          }
        }
        else
        {
          if (k == 0)
          {
            for (int i = 1; i < len; i++)
            {
              int x  = input[i] - '0';
              output += x * pow(10, len - i - 1);
            }
          }
          else
          {
            for (int i = 1; i < k; i++)
            {
              int x  = input[i] - '0';
              output += x * pow(10, k - i - 1);
            }
            for (int i = k + 1; i < len; i++)
            {
              int x = input[i] - '0';
              output += x * pow(10, k - i);
            }
          }
          output = -output;
        }
        return output;
      }

      int FindDecimalPoint(String input)
      {
        for (int i = 0; i < input.length(); i++)
        {
          if (input[i] == '.')
          {
            return i;
          }
        }
        return 0;
      }

      void manual_control()
      {
        while (true)
        {
          irrecv.resume();
          while (!irrecv.decode(&results))
          {
            delay(100);
          }
          Serial.print("IRremote:");
          Serial.println(results.value);
          switch (results.value)
          {
            case 16753245:
              {
                stepper0.setSpeed(MaxSpeed);
```

```
                stepper0.step(STEPS);
                stepCount[0] += 1;
                break;
              }
            case 16720605:
              {
                stepper0.setSpeed(MaxSpeed);
                stepper0.step(-STEPS);
                stepCount[0] -= 1;
                break;
              }
            case 16736925:
              {
                stepper1.setSpeed(MaxSpeed);
                stepper1.step(STEPS);
                stepCount[1] += 1;
                break;
              }
            case 16712445:
              {
                stepper1.setSpeed(MaxSpeed);
                stepper1.step(-STEPS);
                stepCount[1] -= 1;
                break;
              }


            case 16769565:
              {
                stepper2.setSpeed(MaxSpeed);
                stepper2.step(STEPS);
                stepCount[2] += 1;
                break;
              }
            case 16761405:
              {
                stepper2.setSpeed(MaxSpeed);
                stepper2.step(-STEPS);
                stepCount[2] -= 1;
                break;
              }

            case 16769055:
              {
                stepper3.setSpeed(MaxSpeed);
                stepper3.step(STEPS);
                stepCount[3] += 1;
                break;
              }
            case 16738455:
              {
                stepper3.setSpeed(MaxSpeed);
                stepper3.step(-STEPS);
                stepCount[3] -= 1;
                break;
              }

            case 16754775:
              {
```

```
                stepper4.setSpeed(MaxSpeed);
                stepper4.step(STEPS);
                stepCount[4] += 1;
                break;
              }
            case 16750695:
              {
                stepper4.setSpeed(MaxSpeed);
                stepper4.step(-STEPS);
                stepCount[4] -= 1;
                break;
              }
            case 16748655:
              {
                stepper5.setSpeed(MaxSpeed);
                stepper5.step(STEPS);
                stepCount[5] += 1;
                break;
              }
            case 16756815:
              {
                stepper5.setSpeed(MaxSpeed);
                stepper5.step(-STEPS);
                stepCount[5] -= 1;
                break;
              }

            case 16726215:
              {
                mode = 0;
                irrecv.resume();
                return;
              }
          }
          delay(100);
        }
      }

      void manual_control2()
      {
        int CurrentStepperInt;
        while (true)
        {
          lcd.clear();
          lcd.setCursor(0, 0);
          lcd.print("Motor:(1-6?)");
          //lcd.setCursor(0, 1);
          //lcd.print("OK to return");

          irrecv.resume();
          while (!irrecv.decode(&results))
          {
            delay(100);
          }

          switch (results.value)
          {
            case 16753245:
              {
```

```
                CurrentStepperInt = 0;
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Motor1");
                break;
              }
            case 16736925:
              {
                CurrentStepperInt = 1;
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Motor2");
                break;
              }
            case 16769565:
              {
                CurrentStepperInt = 2;
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Motor3");
                break;
              }
            case 16720605:
              {
                CurrentStepperInt = 3;
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Motor4");
                break;
              }
            case 16712445:
              {
                CurrentStepperInt = 4;
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Motor5");
                break;
              }
            case 16761405:
              {
                CurrentStepperInt = 5;
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Motor6");
                break;
              }
            case 16726215:
              {
                Serial.println("Mode0!");
                mode = 0;
                irrecv.resume();
                Serial.println("Return");
                return;
              }
            default:
              {
                Serial.println("Re");
                mode = 1;
```

```
                return;
              }
          }
          int k = manual_control_delay();
          Serial.println(k);
          if (k == 1)
          {
            lcd.setCursor(0, 1);
            lcd.print("Forward");
            manual_control_assist(1, CurrentStepperInt);

          }
          else if (k == 2)
          {
            lcd.setCursor(0, 1);
            lcd.print("Backward");
            manual_control_assist(2, CurrentStepperInt);
          }

        }
      }
      int manual_control_delay()
      {
        irrecv.resume();
        while (!(irrecv.decode(&results)))
        {
          delay(100);
        }
        Serial.println(results.value);
        if (results.value == 16716015)
        {
          irrecv.resume();
          Serial.println("Return1");
          return 1;
        }
        else if (results.value == 16734885)
        {
          irrecv.resume();
          Serial.println("Return2");
          return 2;
        }
        else
        {
          Serial.println("Delay");
          manual_control_delay();
        }
      }

      void manual_control_assist(int k, int CurrentStepperInt)
      {
        double steps;
        double CountPerStep;
        Stepper CurrentStepper = StepperIndex(CurrentStepperInt);
        if (k == 1)
        {
          steps = STEPS / 10;
          CountPerStep = steps / STEPS;
        }
        else if (k == 2)
```

```
        {
          steps = -STEPS / 10;
          CountPerStep = steps / STEPS;
        }
        irrecv.resume();
        while (!(irrecv.decode(&results)))
        {
          Serial.println("Moving");
          CurrentStepper.setSpeed(MaxSpeed);
          CurrentStepper.step(steps);
          stepCount[CurrentStepperInt] += CountPerStep;
        }
        Serial.println(results.value);
        if (k == 1)
        {
          if (results.value == 16716015)
          {
            Serial.println("Return");
            irrecv.resume();
            return;
          }
          else
          {
            Serial.println("Keep Moving");
            irrecv.resume();
            manual_control_assist(k, CurrentStepperInt);
          }
        }
        else if (k == 2)
        {
          if (results.value == 16734885)
          {
            Serial.println("Return");
            irrecv.resume();
            return;
          }
          else
          {
            Serial.println("Keep Moving");
            irrecv.resume();
            manual_control_assist(k, CurrentStepperInt);
          }
        }
      }

      void auto_control()
      {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("xfore");
        auto_control_assist(0, 0);
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("theta1");
        auto_control_assist(1, 4);
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("xrear");
        auto_control_assist(2, 1);
```

```
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("theta2");
    auto_control_assist(3, 5);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Ymid");
    auto_control_assist(4, 3);
    //auto_control_assist(5,2);

    mode = 0;
  }

  void auto_control_assist(int SonsorInt, int MotorInt)
  {
    Stepper CurrentStepper = StepperIndex(MotorInt);
    double steps = STEPS / 40;
    while (true)
    {
      CurrentStepper.setSpeed(MaxSpeed);
      CurrentStepper.step(steps);
      stepCount[MotorInt] += steps / STEPS;
      if (SonsorInt == 4)
      {
        SensorRead(5);
        SensorRead(SonsorInt);
        if (newdata[SonsorInt] < 98 || newdata[5] < 98)
          return;
      }
      else
      {
        SensorRead(SonsorInt);
        if (newdata[SonsorInt] < 98)
          return;
      }
    }
  }

  void SensorRead(int i)
  {
    sensorRead[i] = analogRead(sensor[i]);
    newdata[i] = map(sensorRead[i], 0, 1023, 0, 100);
    Serial.print("Sensor["); Serial.print(i); Serial.print("]:");
    Serial.println(newdata[i]);
  }

  void PrintStep(int i)
  {
    Serial.print("stepCount["); Serial.print(i); Serial.print("]:");
    Serial.println(stepCount[i]);
  }

  //1-16753245
  //2-16736925
  //3-16769565
  //4-16720605
  //5-16712445
  //6-16761405
  //7-16769055
```

```
//8-16754775
//9-16748655
//*-16738455
//0-16750695
//#-16756815
//ok-16726215
//w-16718055
//s-16730805
//a-16716015
//d-16734885
```